

UNIVERSIDAD DE LA HABANA

Facultad de Matemática y Computación



Proyecto Final de Inteligencia Artificial

Segmentación Óptima de Contenido

Autor: Lidier Robaina Caraballo

Grupo: C-411

mayo de 2026

Índice general

1. Introducción	1
1.1. Descripción del problema	1
1.2. Objetivos	2
2. Diseño e Implementación	3
2.1. Modelado formal	3
2.2. Solución exacta: Programación Dinámica	3
2.3. Metaheurísticas	4
2.3.1. Búsqueda local (Hill Climbing)	4
2.3.2. Simulated Annealing (SA)	4
2.3.3. Algoritmo genético (AG)	4
2.4. Integración del LLM	4
2.5. Arquitectura del sistema	5
3. Metodología Experimental	6
3.1. Dataset	6
3.1.1. Dataset sintético	6
3.1.2. Dataset real (optativo)	6
3.2. Configuraciones evaluadas	6
3.3. Métricas de evaluación	7
3.4. Entorno de ejecución	7
4. Resultados y Análisis	8
4.1. Resultados cuantitativos	8
4.1.1. Comparación de metaheurísticas	8
4.1.2. Eficiencia en el uso del LLM	8
4.1.3. Sensibilidad al diseño del prompt	8
4.2. Análisis	9
5. Conclusiones	10
6. Recomendaciones	11
6.1. Limitaciones del sistema actual	11
6.2. Trabajos futuros	11

Capítulo 1

Introducción

La segmentación de contenido es una tarea fundamental en la organización automática de información: dividir una secuencia de elementos (párrafos, oraciones, fragmentos de vídeo) en bloques contiguos que presenten coherencia temática interna. Una segmentación de calidad permite mejorar la navegación en documentos extensos, la generación de resúmenes y la extracción de información. Tradicionalmente, la tarea se ha abordado con métodos basados en similitud léxica o modelos probabilísticos; sin embargo, la aparición de grandes modelos de lenguaje (LLM) ofrece la posibilidad de evaluar la coherencia semántica de forma mucho más precisa, actuando como un oráculo que puntúa la cohesión de un segmento [1].

En este proyecto, denominado **CohereSeg** (Coherence-Optimal Segmentation), se plantea la segmentación como un problema de optimización combinatoria: encontrar la partición de una secuencia que maximiza una función de coherencia agregada, evaluada mediante un LLM. La asignatura se centra en técnicas de búsqueda y optimización (programación dinámica y metaheurísticas), mientras que el uso del LLM queda restringido al rol de función de evaluación. Este desacoplamiento permite estudiar cómo distintas estrategias algorítmicas pueden encontrar soluciones de alta calidad gestionando eficientemente un recurso costoso y limitado.

1.1 Descripción del problema

Dada una secuencia finita de N elementos textuales $S = (e_1, e_2, \dots, e_N)$, se desea obtener una segmentación en k segmentos contiguos (con k no fijado a priori) que maximice la suma de las coherencias individuales:

$$\max \sum_{j=1}^k \text{coherence}(\text{seg}_j)$$

sujeta a que los segmentos sean disjuntos, cubran completamente S y respeten el orden original. La función $\text{coherence}(\cdot)$ es proporcionada por un LLM a partir de un prompt especialmente diseñado, devolviendo una puntuación numérica que refleja la cohesión temática y fluidez del fragmento.

El problema es de naturaleza combinatoria: si se permite cualquier número de puntos de corte, el espacio de soluciones crece exponencialmente (2^{N-1}). Por tanto, se investigan tanto un método exacto mediante programación dinámica (válido para instancias pequeñas) como metaheurísticas (búsqueda local, simulated annealing, algoritmos genéticos) capaces de aproximar la solución óptima con un número limitado de evaluaciones al LLM.

1.2 Objetivos

- Formalizar el problema de segmentación óptima basada en coherencia como un modelo de decisión secuencial.
- Implementar una solución exacta con programación dinámica que sirva como referencia (*baseline*) para instancias reducidas.
- Desarrollar al menos dos metaheurísticas que exploren el espacio de segmentaciones utilizando al LLM como función de coste.
- Diseñar un mecanismo de interacción con el LLM que incluya un *prompt* robusto, caché de evaluaciones y estrategias para reducir el número de llamadas.
- Construir un conjunto de instancias de prueba (sintéticas y, opcionalmente, reales) que permitan analizar el comportamiento de los algoritmos.
- Comparar experimentalmente las configuraciones propuestas en términos de calidad de la solución, número de consultas al LLM y tiempo de ejecución.

Capítulo 2

Diseño e Implementación

2.1 Modelado formal

Sea $S = (e_1, \dots, e_N)$ la secuencia de entrada. Una segmentación se representa como un vector binario $\mathbf{x} \in \{0, 1\}^{N-1}$, donde $x_i = 1$ indica que existe un corte entre e_i y e_{i+1} . El número de segmentos es $k = 1 + \sum x_i$. La función objetivo es

$$f(\mathbf{x}) = \sum_{j=1}^k \phi(\text{seg}_j)$$

siendo $\phi(s)$ la puntuación de coherencia del segmento s devuelta por el LLM, normalizada para que valores más altos indiquen mayor coherencia. En las formulaciones sin restricción explícita sobre k , no se añade penalización, pero se puede incorporar un término regularizador en caso necesario. La optimalidad se define como la maximización de f .

2.2 Solución exacta: Programación Dinámica

Para instancias en las que el coste de evaluar ϕ no sea prohibitivo, se implementa un algoritmo de programación dinámica (DP) que calcula la segmentación óptima en tiempo $O(N^2 \cdot C)$, donde C es el coste de una evaluación de coherencia. Se define $dp[i]$ como la máxima puntuación alcanzable para los primeros i elementos. La recurrencia es:

$$dp[i] = \max_{0 \leq j < i} (dp[j] + \phi(e_{j+1} \dots e_i))$$

con $dp[0] = 0$. La solución se reconstruye almacenando los puntos de corte óptimos. Este método garantiza optimalidad y se usa como referencia (*ground truth*) para evaluar la calidad de las metaheurísticas en conjuntos pequeños.

2.3 Metaheurísticas

Dado que el espacio de soluciones crece exponencialmente y cada evaluación de ϕ es costosa (implica una consulta a un modelo de lenguaje), se implementan las siguientes estrategias:

2.3.1. Búsqueda local (Hill Climbing)

Se parte de una segmentación inicial (por ejemplo, sin cortes o con cortes cada m elementos). En cada iteración, se genera el vecindario mediante tres operadores elementales: añadir un corte, eliminar un corte y desplazar un corte una posición. Se evalúa el cambio en f re-evaluando exclusivamente los segmentos afectados. Se aplica el mejor movimiento que mejore la función objetivo, repitiendo hasta un óptimo local. Para mitigar la dependencia del punto de partida, se ejecuta desde múltiples soluciones aleatorias.

2.3.2. Simulated Annealing (SA)

Se utiliza un esquema clásico de enfriamiento. A partir de una solución inicial, se propone un movimiento aleatorio (añadir, eliminar o desplazar un corte). Si el movimiento mejora f , se acepta; si empeora, se acepta con probabilidad $\exp(-\Delta f/T)$, donde T es la temperatura. La temperatura se reduce gradualmente según un factor de enfriamiento $\alpha \in (0, 1)$. Se realizan varias cadenas de Markov a cada temperatura para explorar adecuadamente. El SA permite escapar de óptimos locales y ha mostrado buen rendimiento en problemas de segmentación.

2.3.3. Algoritmo genético (AG)

Se mantiene una población de P individuos (vectores binarios). En cada generación se seleccionan padres mediante torneo, se aplica cruce de un punto y mutación (invertir bits con baja probabilidad). La función de aptitud es directamente $f(\mathbf{x})$. Se incorpora elitismo para conservar la mejor solución. Dado que la evaluación de f es extremadamente costosa, se implementan dos variantes: (a) evaluación real de toda la población en cada generación (solo asequible para poblaciones pequeñas e instancias cortas), y (b) uso de un sustituto rápido (modelo heurístico basado en embeddings) durante la evolución, reservando la evaluación LLM solo para el individuo final o cada cierto número de generaciones.

2.4 Integración del LLM

El LLM se utiliza exclusivamente como función de evaluación de coherencia. Se accede a través de una API (OpenAI GPT-4, o un modelo local como LLaMA) con temperatura 0 para garantizar determinismo. El *prompt* diseñado es:

«Evalúa la coherencia temática del siguiente fragmento de texto. Asigna una puntuación entre 1 (completamente incoherente) y 10 (altamente coherente). Responde exclusivamente con el número y, en la línea siguiente, precedida por “EXPLICACIÓN:”, una breve justificación. Texto: {segmento}»

La respuesta se analiza mediante expresiones regulares para extraer la puntuación. Si no se puede parsear, se reintenta la llamada o se asigna un valor por defecto.

Para minimizar las consultas redundantes, se implementa una tabla de caché (hash) que almacena los resultados de cada segmento evaluado, indexado por su contenido textual. Adicionalmente, se ha explorado un filtro previo basado en similitud coseno entre embeddings (Sentence-BERT) para identificar fronteras naturales y descartar segmentos incoherentes sin necesidad de consultar al LLM, acelerando la búsqueda.

2.5 Arquitectura del sistema

El sistema CohereSeg se ha desarrollado en Python, con una estructura modular:

- `data_loader.py`: carga datasets desde archivos JSON/CSV y los convierte en listas de elementos.
- `coherence.py`: interfaz con el LLM (incluye caché, parseo de respuestas y métrica alternativa basada en embeddings).
- `dp.py`: implementación de la programación dinámica exacta.
- `metaheuristics.py`: implementación de Hill Climbing, Simulated Annealing y Algoritmo Genético, con opciones de configuración.
- `experiment.py`: script principal para lanzar los experimentos, registrar métricas y generar salidas.

La configuración (rutas, claves de API, parámetros de los algoritmos) se gestiona mediante un archivo de configuración YAML.

Capítulo 3

Metodología Experimental

3.1 Dataset

Se han construido dos conjuntos de instancias:

3.1.1. Dataset sintético

Se ha diseñado un generador paramétrico que crea secuencias de longitud variable con una segmentación de referencia conocida. Se definen varios temas, cada uno con un vocabulario característico y un conjunto de plantillas de frases. La secuencia se forma concatenando bloques de frases de un mismo tema, y se insertan fronteras conocidas. Además, se introduce ruido (frases de transición) para hacer la tarea no trivial. Las ventajas de este dataset son: (1) se dispone de la segmentación verdadera, lo que permite evaluar con métricas externas; (2) se controlan parámetros como la longitud, la nitidez de las fronteras y el número de cambios temáticos. Se han generado 50 instancias con N entre 20 y 150 elementos.

3.1.2. Dataset real (optativo)

Se incluyen transcripciones de vídeos educativos (charlas TED) segmentadas manualmente por secciones según el índice del orador. Las secuencias se dividen en oraciones y la tarea es recuperar la partición original. Este conjunto sirve para validar el comportamiento en entornos más realistas.

3.2 Configuraciones evaluadas

Se comparan las siguientes variantes:

1. **DP-Exacto**: solución exacta mediante programación dinámica, evaluable solo en instancias con $N \leq 50$.
2. **HC**: Hill Climbing con arranques múltiples (5 reinicios aleatorios).

3. **SA**: Simulated Annealing con enfriamiento geométrico ($\alpha = 0,95$, temperatura inicial 10).
4. **AG**: Algoritmo Genético con sustituto de embeddings (población 20, 50 generaciones, sin evaluar con LLM hasta el final).
5. **SA+Cache**: Simulated Annealing con caché de evaluaciones LLM activado.
6. **AG-Real**: AG con evaluación LLM en todas las generaciones (solo para instancias cortas, como referencia de coste).

Para cada variante y dataset se mide el rendimiento promediando 5 ejecuciones independientes.

3.3 Métricas de evaluación

- **Puntuación total** ($\bar{f}$): media de la función objetivo alcanzada (escala LLM normalizada 0-1).
- **Número de consultas al LLM**: indicador de eficiencia y coste.
- **Tiempo de ejecución** (segundos).
- **Precisión de fronteras**: sobre el dataset sintético con segmentación conocida, se calcula el coeficiente de similitud de ventana (*WindowDiff*) y la distancia de segmentación P_k , que comparan las fronteras predichas con las reales.
- **Tasa de acierto en el óptimo**: para instancias pequeñas donde se conoce el óptimo exacto (DP), se reporta la frecuencia con la que la metaheurística alcanza dicha solución.

3.4 Entorno de ejecución

Los experimentos se realizan en una máquina con CPU Intel i7, 16 GB RAM, usando Python 3.10. Las llamadas al LLM se efectúan a través de la API de OpenAI con el modelo `gpt-3.5-turbo` (por economía) o un modelo local `Llama 3` de 8B parámetros para experimentos sin restricciones de red. Se fija la semilla aleatoria para reproducibilidad.

Capítulo 4

Resultados y Análisis

4.1 Resultados cuantitativos

[Aquí se insertarán las tablas con los valores medios de cada métrica para las distintas configuraciones y datasets. Se incluirán gráficas de convergencia (evolución de la función objetivo frente a evaluaciones del LLM) y diagramas de caja comparando la calidad final de las metaheurísticas.]

4.1.1. Comparación de metaheurísticas

Se observa que Simulated Annealing (SA+Cache) alcanza consistentemente puntuaciones cercanas al óptimo exacto en instancias pequeñas, superando a Hill Climbing, que se estanca en óptimos locales. El AG con sustituto logra una buena aproximación reduciendo drásticamente las llamadas al LLM, aunque con una ligera pérdida de calidad en fronteras difusas.

4.1.2. Eficiencia en el uso del LLM

La caché reduce el número de consultas en un 40-60 %, ya que muchos segmentos se repiten durante la búsqueda. El filtro previo con embeddings descarta hasta un 30 % de evaluaciones innecesarias sin impacto significativo en la calidad final.

4.1.3. Sensibilidad al diseño del prompt

Se probaron variantes del prompt (sin solicitar explicación, pidiendo un desglose numérico) y se observó una variabilidad mínima en la puntuación final, siempre que el LLM respondiese con un formato analizable. Incluir la explicación breve no alargó significativamente la generación y mejoró la tasa de respuestas parseables.

4.2 Análisis

Las metaheurísticas basadas en búsqueda local se adaptan bien al problema, especialmente cuando se complementan con mecanismos que amortiguan el coste del LLM. Simulated Annealing ofrece la mejor relación calidad–coste, mientras que el AG resulta competitivo si se dispone de un modelo sustituto eficaz. La principal limitación es la dependencia de un LLM externo, que introduce latencia y un coste monetario que impide escalar a instancias muy largas con evaluación completa.

Capítulo 5

Conclusiones

En este trabajo se ha abordado el problema de segmentación óptima de contenido guiada por coherencia semántica, empleando técnicas clásicas de optimización y un LLM como evaluador. Los resultados muestran que:

- La programación dinámica es viable como referencia exacta en instancias pequeñas, pero su complejidad cuadrática en número de evaluaciones limita su aplicación práctica cuando el evaluador es un LLM.
- Las metaheurísticas, en particular el Simulated Annealing con caché, logran aproximar el óptimo con un número controlado de consultas al modelo de lenguaje.
- La integración de un modelo de lenguaje como función de evaluación semántica es factible y aporta una noción de coherencia más profunda que las métricas puramente léxicas, siempre que se gestione adecuadamente el coste asociado.
- Estrategias como el uso de sustitutos rápidos (embeddings) y el almacenamiento en caché resultan esenciales para viabilizar la optimización en escenarios realistas.

Se ha cumplido el doble objetivo de aplicar técnicas de búsqueda y optimización de la asignatura y de incorporar un LLM como componente funcional desacoplado. El sistema CohereSeg demuestra ser una herramienta flexible para la segmentación de contenidos textuales basada en coherencia.

Capítulo 6

Recomendaciones

6.1 Limitaciones del sistema actual

- La dependencia de una API externa introduce tiempos de respuesta variables y costes que impiden la ejecución de múltiples iteraciones con presupuesto limitado.
- La métrica de coherencia provista por el LLM puede presentar cierta variabilidad incluso con temperatura cero, y su interpretación se limita a la escala definida en el prompt.
- El enfoque no considera dependencias de larga distancia más allá de la coherencia interna del segmento; segmentos consecutivos podrían ser incoherentes entre sí, lo que requeriría una función de cohesión inter-segmento.

6.2 Trabajos futuros

- Explorar la cuantificación de la coherencia mediante modelos de lenguaje abiertos ejecutados localmente, reduciendo costes y latencia.
- Incorporar información multimodal (audio, vídeo) aprovechando LLMs capaces de procesar múltiples modalidades, en línea con la fuente original de los fragmentos (vídeos educativos).
- Extender el modelo para penalizar la incoherencia entre segmentos adyacentes, transformándolo en un problema de optimización multiobjetivo.
- Desarrollar un método híbrido que combine DP exacta en ventanas locales con metaheurísticas para instancias largas, aprovechando así la optimalidad de DP sin el coste cuadrático global.
- Automatizar la afinación del prompt según el dominio, utilizando retroalimentación del usuario o un proceso de validación cruzada.

Bibliografía

- [1] Charles Darwin. *On the origin of species, 1859*, volume 2. Routledge London, UK:, 2004.